

Today we discuss the model of a complete cyber physical system.

Last time, we've seen discrete time modelling.



The state space model for a linear system is given by something like this:

$$\begin{cases} x_{k+1} = Ax_k + Bu_k \\ y_k = Cx_k + Du_k \end{cases}$$

Typically, the input in a sampling interval, is kept constant.

The value of an interval could be $T = 1\text{ ms}$

Typically, most of physical systems live in continuous time, and then from them we derive the discrete time model, used by computers.

Another important thing we saw is given by the change of coordinates.

One specific state space is the state space model in natural coordinates. If the model is linear, it's described by the A, B, C, D matrices. Sometimes, however, it's useful to do a change of coordinates, done with $z = (T)^{-1}x$, where T is the transformation matrix, to the $^{-1}$ to stress the fact the matrix must be invertible.

Yesterday we've seen a trivial transformation in which we flipped the order of the variables, writing the state x in new coordinates z . We do this as sometimes the analysis of the system could be much simpler in the transformed case. One typical analysis problem is to compute the solution of the system.

Another case is given by the solution of some design problems, like formulation of a good attack, which is simple.

In the new coordinates, we have a system described with new matrices, which is however still the same system. So

$$(\tilde{A}, \tilde{B}, \tilde{C}, \tilde{D}) = (T^{-1}AT, T^{-1}B, CT, D)$$

They are equivalent, which means they describe the same system, and also the two state space models have the same solution.

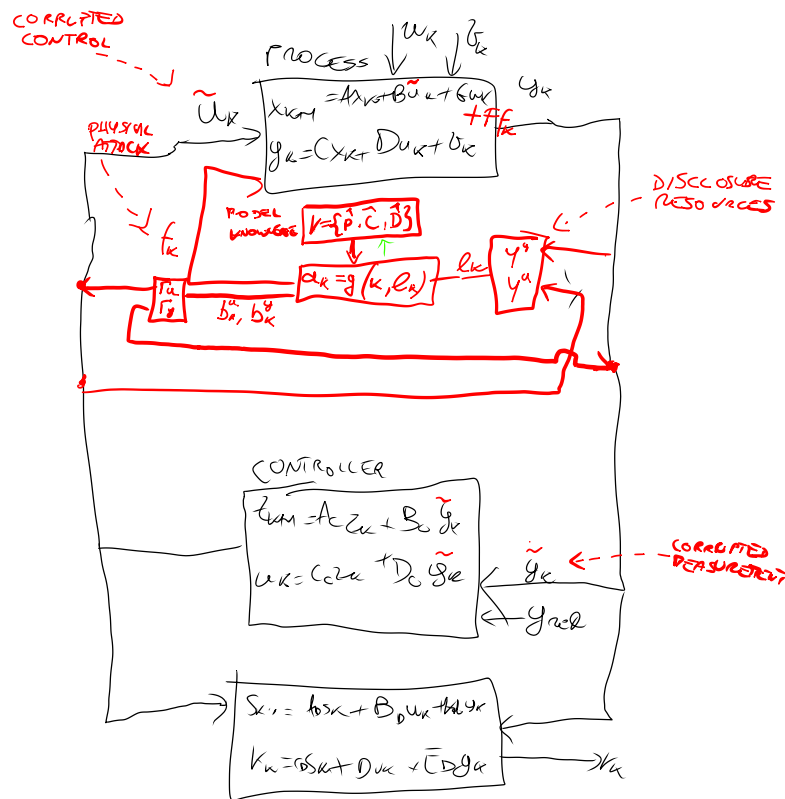
Yesterday we used $T^{-1} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ as an example of invertible matrix. If we don't use an invertible matrix, we may lose information.

After we've reached the new coordinates, we perform the analysis in the new z

coordinates, and then see the result of the analysis in the natural coordinates.

T has to be invertible as we need to be able to go to the original coordinates, such that $x = Tz$.

General model/architecture of a networked CPS



We describe the process with a linear state space model in discrete time.

We'll have:

$$\begin{cases} x_{k+1} = Ax_k + Bu_k + Gw_k \\ y_k = Cx_k + Du_k + v_k \end{cases}$$

w is an external disturbance. v is measurement noise.

We then have the controller. It's a dynamical system, and can be represented with the exact same equations. We call the state of the controller as z .

$$\begin{cases} z_{k+1} = A_c z_k + B_c y_k + B'_c y_{ref} \\ u_k = C_c z_k + D_c y_k + D'_c y_{ref} \end{cases}$$

The input of the controller is the output of the system, y . The output of the controller is the input of the system, u .

Another input for the controller is given by y_{ref} , so we should add to the equations y_{ref} .

We also have the detector.

$$\begin{cases} s_{k+1} = A_d s_k + B_d u_k + K_d y_k \\ r_k = C_d s_k + D_d u_k + E_d y_k \end{cases}$$

This is the scheme of a CPS in absence of attacks.

Let's distinguish two terminologies:

- Normal operation: no attacks, but presence of disturbances, noise, model uncertainty. What is model uncertainty? Given a complex system, we cannot expect to find perfect models, but a good approximation (of how the system works).
 - $y_k \approx y_k^{ref}$, output track reference if controller is good.
 - $r_k \approx 0$. We have no attack alarm raised in this case. Remember the threshold based detection.

We're going to take a look at the idea of consistency of detectors: A consistent detector is one which must not raise false alarms.

- Nominal operations: no attack, no noise, no disturbance, no model uncertainty (perfect model).

Let's now introduce the attacker in the system.

Remember that the attacker is modelled as the set of disclosure resources, disruption resources and model knowledge.

Let's start with disclosure resources. These are the set of output and input variables that the attacker can read in real time.

Check the drawing, red parts for the attacker.

- y_k are the measurements, specifically being a vector of dimension n_y ($y_k \in \mathbb{R}^{n_y}$).
- u_k is a vector of dimension n_u ($u_k \in \mathbb{R}^{n_u}$)

Typically, the attacker has access only to a subset of y_k and u_k .

$$R^y \subseteq \underbrace{\{1, 2, \dots, n_y\}}_{\text{all measurements}} \quad \text{set of measurements the attacker can read}$$

$$R^u \subseteq \underbrace{\{1, 2, \dots, n_u\}}_{\text{all controls}} \quad \text{set of controls the attacker can read in real time}$$

We have Selection matrices.

$|R^y|$ is the number of measurements that the attacker reads.

$|R^u|$ is the number of controls that the attacker reads.

We always have this: $|R^y| \leq n_y, |R^u| \leq n_u$.

≡ Example

$n_y = 5$, and $n_u = 4$. Let's assume $R^y = \{1, 3\}$ and $R^u = \{2, 4\}$.

How to model the attacker reading only specific variables?

We introduce Selection matrices.

We have Y^y and Y^u .

$Y^y = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{pmatrix}$ out of total of 5 outputs it selects the first and third ones

Selection matrices are used to extract only the variables the attacker is reading.

Y^u is defined similarly, to select only the controls attacker can read.

$$Y^u = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

For the example above.

$$\begin{cases} Y^u \cdot u_k & \text{are the controls attacker can read} \\ Y^y \cdot y_k \end{cases}$$

Accumulation of signals (u_k, y_k) ready by attacker from the system, from an initial time k_0 (when the attacker starts reading), up to the current generic time k .

We define l_k as the set of reading up to time k .

Equations for l_k :

$$\begin{cases} l_{k_0} = \{\} \\ l_k = l_{k-1} \cup \begin{pmatrix} Y^y & 0 \\ 0 & Y^u \end{pmatrix} \begin{pmatrix} y_k \\ u_k \end{pmatrix} \end{cases}$$

l_{k_0} is empty at first. l_{k-1} is what the attacker has read at time $k-1$, and the rest is what the attacker reads now.

Now, let's see model knowledge. This is the knowledge that attacker has about the model of the cyber physical system. So, models of the process, controller and detector.

We call the model knowledge $K = \{\hat{P}, \hat{C}, \hat{D}\}$.

- $\hat{P}$ is an approximated model of P .

- $\hat{C}$ similar.
- $\hat{D}$ similar.

We can have three possible cases:

- $K = \{\}$, zero model knowledge, the attacker knows nothing about the system.
- $K = \{\hat{P}, \hat{C}, \hat{D}\}$, partial model knowledge. Attacker knows something about the system.
- $K = \{P, C, D\}$, full model knowledge.

If we have full model knowledge, the attack is simple to design.

Real case, though, is either zero knowledge or partial. Thus, the attack design is difficult to carry out.

Let's see the model of attack logic.

We're going to take a look not at physical attacks f_k , rather at b_k^u, b_k^y , which are

$$a_k = \begin{pmatrix} f_k \\ b_k^u \\ b_k^y \end{pmatrix}$$

Manipulating b^u and b^y stands for a data deception attack (bias injected in u_k and y_k).

Now, let's see the model of disruption resources.

One part is given by the physical parts of system that attacker can target. This is captured by the matrix F , telling us which equations of the system are modified by a physical attack. Taking the name of physical disruption resources.

For us, disruption resources are simply the sets of measurements and controls that the attacker can alter in real time. These are data deception disruption resources.

We can identify two sets:

- $R_I^u \subseteq \{1, 2, \dots, n_u\}$ set of controls the attacker can modify in real time
- $R_I^y \subseteq \{1, 2, \dots, n_y\}$

Typically, an attacker can alter only a small number of commands and measurements.

Again, we need selection matrices.

≡ Example

$$n_y = 5, n_u = 4, R_I^y = \{2, 3\}, R_I^u = \{1, 4\}$$

Notice that values with R are variables that attacker can corrupt.

b vectors are the vectors of corruptions injected into variables.

$b_k^y \in \mathbb{R}^2$, as we can corrupt two measurements, so $b_k^y \in \mathbb{R}^{|R_I^y|}$, and $b_k^u \in \mathbb{R}^{|R_I^u|}$.

In the example, we can have:

$$b_k^y = \begin{pmatrix} b_{2,k}^y \\ b_{3,k}^y \end{pmatrix}$$

$b_{2,k}^y$ is the bias injected by the attacker into output variable 2 at time k ,

The selection matrices for disruption are called Γ . Thus, we have Γ^y and Γ^u .

$$\Gamma^y = \begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \end{pmatrix}$$

Then, let's see the data deception equation, to model the corrupted measurements.

$$\underbrace{y_k}_{\text{original meas.}} + \underbrace{\Gamma^y b_k^y}_{\text{bias injected by attacker}} = \tilde{y}$$

In the example, we had n_y and $R_I^y = \{2, 3\}$, corrupted with bias b_k^y .

$$\begin{pmatrix} y_{1,k} \\ y_{2,k} \\ y_{3,k} \\ y_{4,k} \\ y_{5,k} \end{pmatrix} + \begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} b_{2,k}^y \\ b_{3,k}^y \end{pmatrix} = \begin{pmatrix} y_{1,k} \\ y_{2,k} + b_{2,k}^y \\ y_{3,k} + b_{3,k}^y \\ y_{4,k} \\ y_{5,k} \end{pmatrix}$$

For controls, we have:

- $u_k + \Gamma_I^u b_k^u = \tilde{u}_k$, these are the corrupted commands.

Γ_I^y is a $n_y \times |R_I^y|$, while Γ_I^u is a $n_u \times |R_I^u|$, matrix of 0s and 1s.

Notice that now u arriving at the plant might be corrected, having $\tilde{u}$, which will be the value used in the equations, as well as having y corrupted, $\tilde{y}$, having the

corrupted value in the equations of the controller.

The attacker both performs direct interventions, on the network channel, modifying u and y . There are, however, also indirect interventions, for example the value outputted by the controller, u , will be altered, due to the presence of corrupted data $\tilde{y}$.

Model knowledge is typically built, progressively, by analysing data. Through specific algorithms, the attacker can progressively build the knowledge of the model of the system (green arrow in drawing).

Now, let's see how to model typical classes of attacks, using the above architecture.

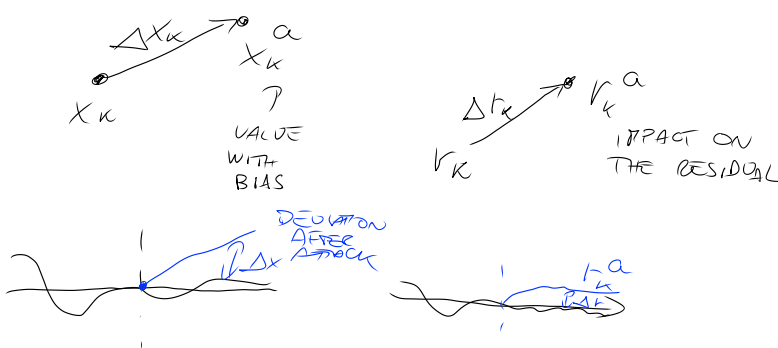
The first trivial attack is the Eavesdropping attack. Attacker accumulates data by reading from measurements and control channels. This attack does not require no model knowledge and no disruption resources, but we need disclosure resources R^u, R^y .

It's an important attack as through this we can gain knowledge and build the model.

For the second attack, let's discuss the bias injection attack. With this attack, the attack aims at introducing constant bias in channels. In FDIA, in the other hand, biases change over time.

The attack goal is to cause damage, while remaining undetected.

The attack problem is to find the variables to corrupt, and the bias to inject.



We have the impact on the attack for x given by Δx , and for r as Δr . They tell us the deviation from the expected values, in continuous time, of x and r .

How do we design optimal bias injection attacks?

Through optimisation problem, like

$$\max_{b_k^u, b_k^y} ||\Delta X_k||$$

I want to make the system deviate as much as possible.

The value of $||\Delta x_k||$ stands for the norm, which is the length of deviation vector.

The whole thing is subject to

$$||\Delta r_k|| \approx 0$$

Other constraints are given by:

- the model of the system.
- limitations of attacker.

Thus typically some attacks are then formulated as optimization problems, requiring to maximize the impact on the state.

In the target function, we could add $-||b_k||$.

So, an optimal attack is one that has:

- large impact on x (damage)
- small impact on r (undetectability)
- inject small biases
- is sparse (corrupts few measurements and control), simpler to carry out.

To be able to compute the optimal attack, the attacker will need model knowledge, necessary to compute Δx and Δr from the biases injected, but a simplified one would suffice. Disclosure resources are not necessary, however we need disruption resources.

So, to recap, they can be performed by solving optimization problem, with the goal to make the attack impactful and undetectable and simple.